Лабораторная работа №4

Линейная алгебра

Умед Джамшедович Назармамадов

В отчёте описано изучение возможностей специализированных пакетов Julia (`Statistics`, `LinearAlgebra`, `BenchmarkTools`) для выполнения и оценки эффективности операций над объектами линейной алгебры.

Воспроизведены примеры из методических указаний: поэлементные операции над массивами, транспонирование, ранг, определитель и обратная матрица, нормы и повороты, матричное умножение, LU-, QR- и спектральное разложения, работа с матрицами специальной структуры и рациональными матрицами. Выполнены задания для самостоятельной работы: произведения векторов, системы линейных уравнений, диагонализация матриц, функции от матриц, линейные модели экономики.

Все результаты подтверждены скриншотами выполнения кода в Jupyter Lab.

Содержание

1 Введение **7**
1.1 Актуальность 7
1.2 Цель работы 7
1.3 Задачи 7
1.4 Объект и предмет исследования 8
1.5 Методы исследования 8
1.6 Схема выполнения работы 8

2 Теоретическая часть **10**

3 Материалы и методы **12**

4 Выполнение лабораторной работы **13**
4.1 Подготовка рабочего пространства 13
4.2 Поэлементные операции над многомерными массивами (раздел 4.2.1) 14
4.3 Транспонирование, след, ранг, определитель и инверсия матрицы (раздел 4.2.2) 15
4.4 Нормы векторов и матриц, повороты (раздел 4.2.3) 16
4.5 Матричное умножение, единичная матрица, скалярное произведение (раздел 4.2.4) 18
4.6 Факторизация. Специальные матричные структуры (раздел 4.2.5) . . 19
4.7 Общая линейная алгебра (раздел 4.2.6) 25
4.8 Задания для самостоятельной работы (раздел 4.4) 26

5 Результаты **37**

6 Обсуждение результатов **39**

7 Заключение **41**

Список литературы **42**

A Листинг кода блокнота linear-algebra.ipynb **43**

Список иллюстраций

1.1 Блок-схема выполнения лабораторной работы 9

4.1 Создание ветки feature/lab04 13
4.2 Создание каталога labs/lab04/project 13
4.3 Запуск Jupyter Lab 14
4.4 Матрица 4×3, суммы и начало вычисления произведений 14
4.5 Произведения по строкам, подключение пакета Statistics, средние значения 15
4.6 Транспонирование, след, диагональ и ранг матрицы 16
4.7 Обратная и псевдообратная матрицы, определитель, нормы, расстояние и угол между векторами 17
4.8 Нормы матрицы, повороты и переворачивания 18
4.9 Скалярное произведение, решение СЛАУ и LU-факторизация 19
4.10 Составные части LU-факторизации, решение СЛАУ и определитель . 20
4.11 QR-факторизация и спектральное разложение симметризованной матрицы 21
4.12 Проверка спектрального разложения, подключение BenchmarkTools, матрица 1000×1000 22
4.13 Симметризованная матрица 1000×1000 22
4.14 Проверка симметричности, добавление шума, явное указание структуры Symmetric 23
4.15 Время вычисления собственных значений: симметричная, зашумлённая и явно симметричная матрицы 24
4.16 Трёхдиагональная матрица $10^6 \times 10^6$ и время вычисления eigmax . . . 25
4.17 Решение СЛАУ и LU-разложение для матрицы с рациональными элементами 26
4.18 Скалярное и внешнее произведение вектора на себя 27
4.19 Функция solve_slau и системы с двумя неизвестными (a)–(c) 28
4.20 Системы с двумя неизвестными (d)–(f) 29
4.21 Системы с тремя неизвестными (a)–(d) 29
4.22 Приведение матриц к диагональному виду 30
4.23 Степень, квадратный и кубический корни матриц 31
4.24 Квадратный корень из матрицы с отрицательным собственным значением 32
4.25 Собственные значения матрицы 5×5, диагональная и нижнетреугольная матрицы, оценка времени 33

4.26 Проверка продуктивности по определению 34
4.27 Критерий продуктивности через матрицу $(E - A)^{-1}$ 35
4.28 Спектральный критерий продуктивности 36

Список таблиц

2.1 Основные функции, используемые в работе 11

5.1 Результаты решения систем линейных уравнений 38

1 Введение

1.1 Актуальность

Линейная алгебра лежит в основе большинства методов статистического анализа данных: регрессии, метода главных компонент, решения систем уравнений, моделирования динамических и экономических систем. Julia содержит встроенную поддержку линейной алгебры через стандартный пакет `LinearAlgebra`, использующий библиотеки BLAS и LAPACK, поэтому умение выбирать подходящие функции и типы матриц напрямую влияет на скорость и надёжность вычислений.

1.2 Цель работы

Изучить возможности специализированных пакетов Julia для выполнения и оценки эффективности операций над объектами линейной алгебры.

1.3 Задачи

1. Используя Jupyter Lab, повторить примеры из раздела 4.2 методических указаний.
2. Выполнить задания для самостоятельной работы (раздел 4.4):
    - произведения векторов (скалярное и внешнее);

- решение систем линейных алгебраических уравнений с двумя и тремя неизвестными;
- приведение матриц к диагональному виду, вычисление степеней и корней матриц, оценка эффективности операций над матрицей 5×5;
- проверка продуктивности матриц в линейной модели экономики.

1.4 Объект и предмет исследования

Объект исследования — пакеты Julia `Statistics`, `LinearAlgebra` и `BenchmarkTools`. Предмет исследования — операции над векторами и матрицами: поэлементные операции, факторизации, решение СЛАУ, собственные значения, работа с матрицами специальной структуры, оценка времени выполнения.

1.5 Методы исследования

Практическая работа в интерактивной среде Jupyter Lab (ядро IJulia): последовательное выполнение примеров и заданий в ячейках блокнота, анализ полученных результатов, измерение времени выполнения макросом `@btime`.

1.6 Схема выполнения работы

На рис. -Рисунок 1.1 представлена общая последовательность действий при выполнении работы.



Рисунок 1.1: Блок-схема выполнения лабораторной работы

2 Теоретическая часть

Нормы. Евклидова норма вектора $\vec{x}$ — $\|\vec{x}\|_2 = \sqrt{x_1^2 + \cdots + x_n^2}$, p-норма — $\|\vec{x}\|_p = \left(\sum_{i=1}^{n} |x_i|^p\right)^{1/p}$. Евклидово расстояние между векторами — $\|\vec{x} - \vec{y}\|_2$, угол между ними — $\arccos \dfrac{\vec{x}^T \vec{y}}{\|\vec{x}\|_2 \|\vec{y}\|_2}$.

Факторизации. Факторизация (разложение) матрицы — её представление в виде произведения матриц специального вида:

- LU-разложение: $A = LU$, где L — нижняя, U — верхняя треугольная матрица; с перестановкой строк — LUP-разложение $PA = LU$;
- QR-разложение: $A = QR$, где Q — ортогональная ($Q^T Q = I$), R — верхняя треугольная матрица;
- спектральное разложение: $A = V \Lambda V^{-1}$, где столбцы V — собственные векторы, Λ — диагональная матрица собственных значений.

Матрицы специальной структуры. Явное указание структуры (`Symmetric`, `Diagonal`, `LowerTriangular`, `SymTridiagonal` и др.) позволяет Julia использовать специализированные, более быстрые алгоритмы и хранить только существенные элементы матрицы.

Линейная модель экономики записывается как СЛАУ $x - Ax = y$, где элементы матрицы A и векторов x, y неотрицательны. Матрица A называется продуктивной, если решение x при любой неотрицательной правой части y неотрицательно. Эквивалентные критерии продуктивности:

- матрица $(E - A)^{-1}$ имеет только неотрицательные элементы;

- все собственные значения матрицы A по модулю меньше 1 (спектральный критерий).

Основные использованные функции перечислены в табл. -Таблица 2.1.

Таблица 2.1: Основные функции, используемые в работе

Функция	Назначение	Пакет
`sum`, `prod`, `mean`	сумма, произведение, среднее (в том числе по осям `dims`)	`Base`, `Statistics`
`transpose`, `tr`, `diag`, `rank`, `inv`, `det`, `pinv`	транспонирование, след, диагональ, ранг, обратная, определитель, псевдообратная	`LinearAlgebra`
`norm`, `opnorm`	норма вектора, норма матрицы	`LinearAlgebra`
`lu`, `qr`, `eigen`, `eigvals`	LU-, QR-, спектральное разложение, собственные значения	`LinearAlgebra`
`Symmetric`, `Diagonal`, `LowerTriangular`, `SymTridiagonal`	типы матриц специальной структуры	`LinearAlgebra`
`@btime`	измерение времени выполнения	`BenchmarkTools`

3 Материалы и методы

Работа выполнена в следующем окружении:

- ОС: WSL2 Ubuntu (Windows);
- Julia 1.10, Jupyter Lab, ядро IJulia;
- пакеты: `Statistics` (v1.10.0), `LinearAlgebra` (стандартная библиотека), `BenchmarkTools` (v1.8.0);
- рабочее пространство организовано по шаблону курса, лабораторная выполнена в ветке `feature/lab04` git-репозитория `2026-2--study--computer-practic` блокнот — `labs/lab04/project/linear-algebra.ipynb`.

Последовательность действий: создание ветки через git-flow, запуск Jupyter Lab, создание блокнота с ядром Julia, последовательное выполнение примеров раздела 4.2 и заданий раздела 4.4 с фиксацией результатов скриншотами.

4 Выполнение лабораторной работы

4.1 Подготовка рабочего пространства

Для лабораторной работы создана ветка `feature/lab04` командой `git flow feature start lab04` (рис. -Рисунок 4.1).

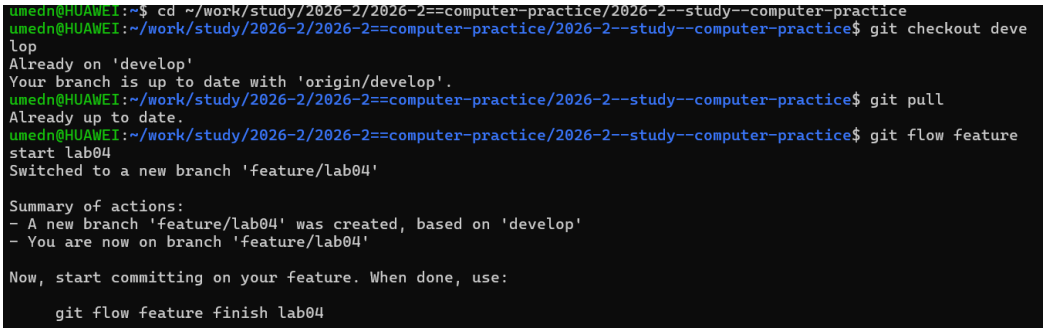


Рисунок 4.1: Создание ветки feature/lab04

Создан каталог для проекта лабораторной работы `labs/lab04/project` (рис. -Рисунок 4.2).

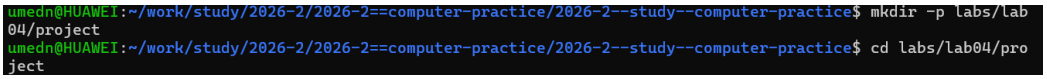


Рисунок 4.2: Создание каталога labs/lab04/project

Запущен Jupyter Lab (рис. -Рисунок 4.3).

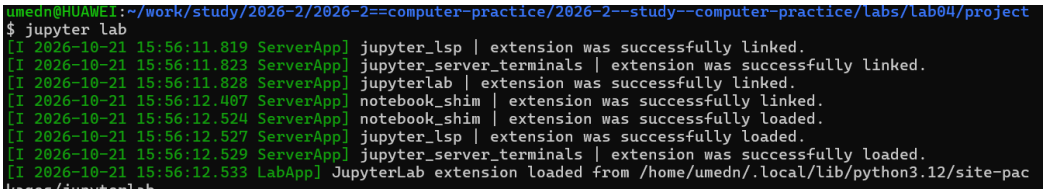


Рисунок 4.3: Запуск Jupyter Lab

4.2 Поэлементные операции над многомерными массивами (раздел 4.2.1)

В блокноте создана матрица `a` размера 4×3 со случайными целыми числами от 1 до 20. Вычислены поэлементная сумма всех элементов, суммы по столбцам (`dims=1`) и по строкам (`dims=2`), а также поэлементное произведение (рис. -Рисунок 4.4).

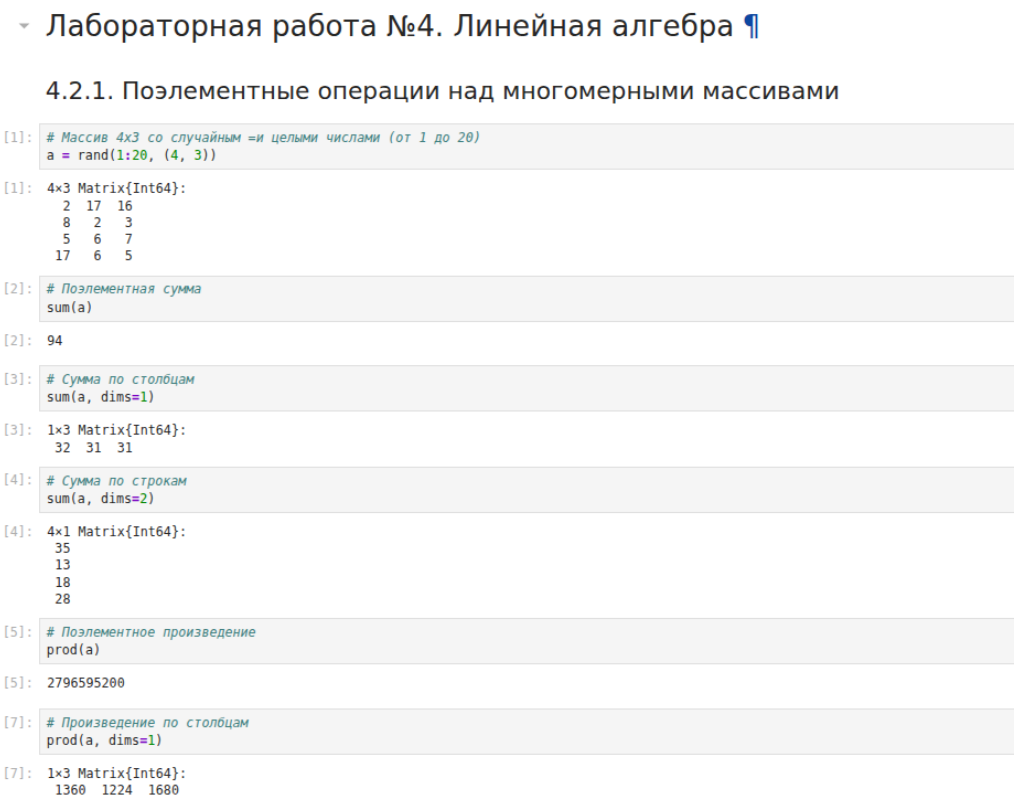


Рисунок 4.4: Матрица 4×3, суммы и начало вычисления произведений

Далее вычислены произведения по строкам. Для работы со средними значениями подключён пакет `Statistics`, вычислены среднее значение массива, средние по столбцам и по строкам (рис. -Рисунок 4.5).

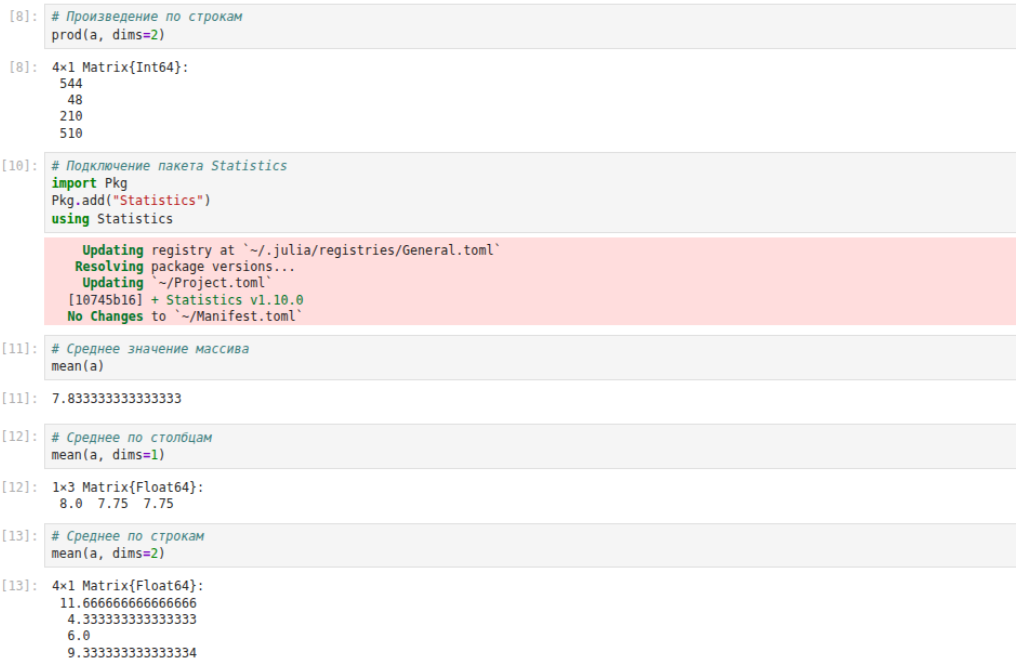


Рисунок 4.5: Произведения по строкам, подключение пакета Statistics, средние значения

4.3 Транспонирование, след, ранг, определитель и инверсия матрицы (раздел 4.2.2)

Подключён пакет `LinearAlgebra`, создана матрица `b` размера 4×4. Выполнены транспонирование, вычисление следа, извлечение диагонали, вычисление ранга (рис. -Рисунок 4.6).

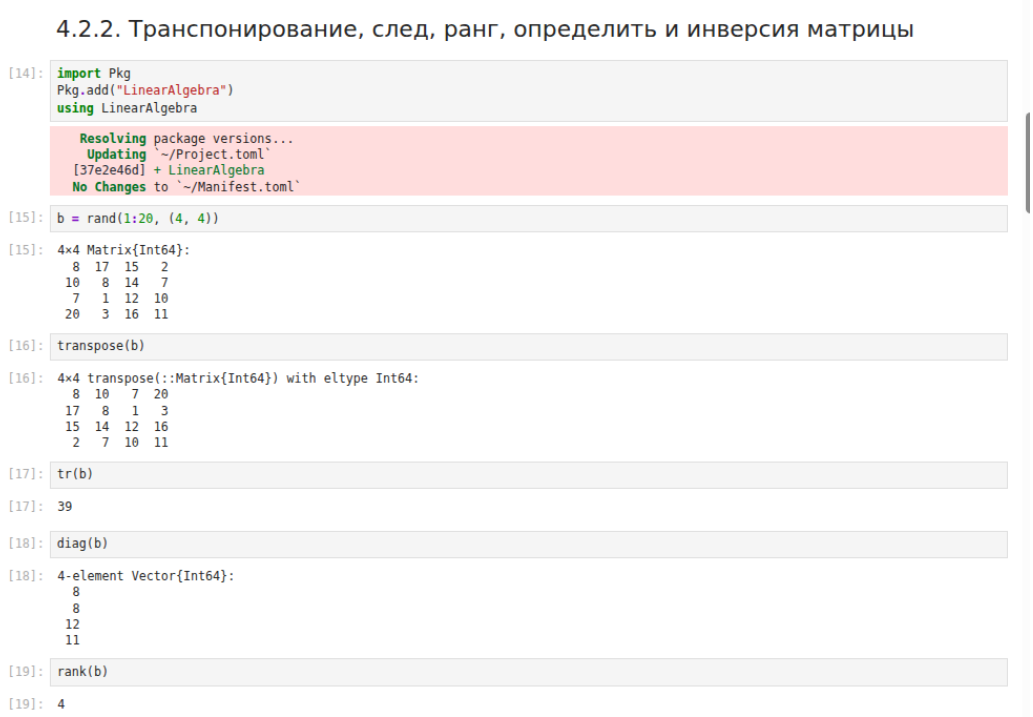


Рисунок 4.6: Транспонирование, след, диагональ и ранг матрицы

Вычислены обратная матрица `inv(b)`, определитель `det(b)` и псевдообратная матрица `pinv(b)` (рис. -Рисунок 4.7). Для квадратной невырожденной матрицы обратная и псевдообратная матрицы совпадают.

4.4 Нормы векторов и матриц, повороты (раздел 4.2.3)

Для вектора $X = (2, 4, -5)$ вычислены евклидова норма и p-норма при $p = 1$. Евклидово расстояние между векторами X и $Y = (1, -1, 3)$ найдено функцией `norm(X - Y)` и проверено по базовому определению `sqrt(sum((X - Y).^2))` — результаты совпали. Вычислен угол между векторами (рис. -Рисунок 4.7).

```
[20]: inv(b)

[20]: 4×4 Matrix{Float64}:
 -0.358974   0.846154  -0.435897  -0.0769231
 -2.0        5.0       -2.0       -1.0
  2.93407   -7.10989    2.87912    1.37363
 -3.0696     7.43956   -2.84982   -1.49451

[21]: det(b)

[21]: 272.999999999997

[22]: pinv(b)

[22]: 4×4 Matrix{Float64}:
 -0.358974   0.846154  -0.435897  -0.0769231
 -2.0        5.0       -2.0       -1.0
  2.93407   -7.10989    2.87912    1.37363
 -3.0696     7.43956   -2.84982   -1.49451
```

4.2.3. Вычисление нормы векторов и матриц, повороты, вращения

```
[23]: X = [2, 4, -5]
      norm(X)

[23]: 6.708203932499369

[24]: p = 1
      norm(X, p)

[24]: 11.0

[25]: X = [2, 4, -5]
      Y = [1, -1, 3]
      norm(X - Y)

[25]: 9.486832980505138

[26]: sqrt(sum((X - Y) .^ 2))

[26]: 9.486832980505138

[27]: acos((transpose(X) * Y) / (norm(X) * norm(Y)))

[27]: 2.4404307889469252
```

Рисунок 4.7: Обратная и псевдообратная матрицы, определитель, нормы, расстояние и угол между векторами

Для матрицы `d` вычислены евклидова норма и p-норма (`opnorm`), выполнены поворот на 180 градусов (`rot180`), переворачивание строк и столбцов (`reverse`) (рис. -Рисунок 4.8).

```
[28]: d = [5 -4 2; -1 2 3; -2 1 0]
      opnorm(d)

[28]: 7.147682841795258

[29]: p = 1
      opnorm(d, p)

[29]: 8.0

[30]: rot180(d)

[30]: 3×3 Matrix{Int64}:
       0   1  -2
       3   2  -1
       2  -4   5

[31]: reverse(d, dims=1)

[31]: 3×3 Matrix{Int64}:
      -2   1  0
      -1   2  3
       5  -4  2

[32]: reverse(d, dims=2)

[32]: 3×3 Matrix{Int64}:
       2  -4   5
       3   2  -1
       0   1  -2
```

4.2.4. Матричное умножение, единичная матрица, скалярное произведение

```
[33]: A = rand(1:10, (2, 3))
      B = rand(1:10, (3, 4))
      A * B

[33]: 2×4 Matrix{Int64}:
       154  98  135  113
        84  54   55   71

[34]: Matrix{Int}(I, 3, 3)

[34]: 3×3 Matrix{Int64}:
       1  0  0
       0  1  0
       0  0  1
```

Рисунок 4.8: Нормы матрицы, повороты и переворачивания

4.5 Матричное умножение, единичная матрица, скалярное произведение (раздел 4.2.4)

Созданы матрицы A (2×3) и B (3×4), вычислено произведение `A * B` — матрица 2×4 (рис. -Рисунок 4.8). Создана единичная матрица 3×3 (`Matrix{Int}(I, 3, 3)`). Скалярное произведение векторов X и Y вычислено двумя способами: `dot(X, Y)` и `X' * Y` — оба дали значение -17 (рис. -Рисунок 4.9).

4.6 Факторизация. Специальные матричные структуры (раздел 4.2.5)

Задана случайная матрица A размера 3×3, единичный вектор x и вектор $b = Ax$. Решение системы $Ax = b$ оператором \ вернуло вектор, равный единичному с точностью до погрешности вычислений. Выполнено LU-факторизация `lu(A)` (рис. -Рисунок 4.9).

```
[35]: X = [2, 4, -5]
      Y = [1, -1, 3]
      dot(X, Y)

[35]: -17

[36]: X' * Y

[36]: -17
```

4.2.5. Факторизация. Специальные матричные структуры

```
[37]: A = rand(3, 3)

[37]: 3×3 Matrix{Float64}:
       0.746873  0.0195377  0.579368
       0.132298  0.176009   0.868318
       0.18203   0.136138   0.427721

[38]: x = fill(1.0, 3)

[38]: 3-element Vector{Float64}:
       1.0
       1.0
       1.0

[39]: b = A * x

[39]: 3-element Vector{Float64}:
       1.3457792015792678
       1.1766243067500157
       0.74588862513469

[40]: A \ b

[40]: 3-element Vector{Float64}:
       1.0
       1.000000000000001
       0.9999999999999998

[42]: Alu = lu(A)

[42]: LU{Float64, Matrix{Float64}, Vector{Int64}}
      L factor:
      3×3 Matrix{Float64}:
       1.0       0.0       0.0
       0.177136  1.0       0.0
       0.243723  0.761387  1.0
      U factor:
      3×3 Matrix{Float64}:
       0.746873  0.0195377   0.579368
       0.0       0.172548    0.765691
       0.0       0.0        -0.296472
```

Рисунок 4.9: Скалярное произведение, решение СЛАУ и LU-факторизация

Из объекта факторизации извлечены вектор перестановок, матрицы L и U. Система решена двумя способами — через матрицу A и через объект факторизации `Alu`, определитель вычислен через `det(A)` и `det(Alu)`; результаты совпадают (рис. -Рисунок 4.10).

```
[44]: Alu.p

[44]: 3-element Vector{Int64}:
       1
       2
       3

[45]: Alu.L

[45]: 3×3 Matrix{Float64}:
       1.0       0.0       0.0
       0.177136  1.0       0.0
       0.243723  0.761387  1.0

[46]: Alu.U

[46]: 3×3 Matrix{Float64}:
       0.746873  0.0195377   0.579368
       0.0       0.172548    0.765691
       0.0       0.0        -0.296472

[47]: A \ b

[47]: 3-element Vector{Float64}:
       1.0
       1.000000000000001
       0.9999999999999998

[48]: Alu \ b

[48]: 3-element Vector{Float64}:
       1.0
       1.000000000000001
       0.9999999999999998

[49]: det(A)

[49]: -0.038206708473220455

[50]: det(Alu)

[50]: -0.038206708473220455
```

Рисунок 4.10: Составные части LU-факторизации, решение СЛАУ и определитель

Выполнена QR-факторизация `qr(A)`, извлечены матрицы Q и R, проверена ортогональность матрицы Q (произведение $Q^T Q$ равно единичной матрице с точностью до погрешности порядка 10^{-16}). Матрица симметризована ($A_{sym} = A + A^T$) и для неё выполнено спектральное разложение `eigen(Asym)` (рис. -Рисунок 4.11).

```
[53]: Aqr = qr(A)

[53]: LinearAlgebra.QRCompactWY{Float64, Matrix{Float64}, Matrix{Float64}}
      Q factor: 3×3 LinearAlgebra.QRCompactWYQ{Float64, Matrix{Float64}, Matrix{Float64}}
      R factor:
      3×3 Matrix{Float64}:
       -0.780037  -0.0803282  -0.80182
        0.0       -0.208427   -0.757923
        0.0        0.0        -0.235002

[54]: Aqr.Q

[54]: 3×3 LinearAlgebra.QRCompactWYQ{Float64, Matrix{Float64}, Matrix{Float64}}

[55]: Aqr.R

[55]: 3×3 Matrix{Float64}:
       -0.780037  -0.0803282  -0.80182
        0.0       -0.208427   -0.757923
        0.0        0.0        -0.235002

[56]: Aqr.Q' * Aqr.Q

[56]: 3×3 Matrix{Float64}:
         1.0          0.0          0.0
         0.0          1.0          1.11022e-16
        -2.77556e-17  1.11022e-16  1.0

[57]: Asym = A + A'

[57]: 3×3 Matrix{Float64}:
       1.49375   0.151836  0.761398
       0.151836  0.352018  1.00446
       0.761398  1.00446   0.855442

[58]: AsymEig = eigen(Asym)

[58]: Eigen{Float64, Float64, Matrix{Float64}, Vector{Float64}}
      values:
      3-element Vector{Float64}:
       -0.5005053997462583
        0.9173292542167584
        2.28438191273639
      vectors:
      3×3 Matrix{Float64}:
       -0.192979   0.708862  -0.678435
       -0.733498  -0.563474  -0.380104
        0.651721  -0.424278  -0.628687
```

Рисунок 4.11: QR-факторизация и спектральное разложение симметризованной матрицы

Проверка `inv(AsymEig) * Asym` даёт единичную матрицу. Для оценки эффективности подключён пакет `BenchmarkTools`, создана случайная матрица 1000×1000 (рис. -Рисунок 4.12, рис. -Рисунок 4.13).

```
[61]: inv(AsymEig) * Asym
```

```
[61]: 3×3 Matrix{Float64}:
        1.0           3.60822e-16  -4.57967e-16
        2.88658e-15   1.0          -4.88498e-15
       -1.94289e-15  -2.33147e-15   1.0
```

```
[62]: import Pkg
      Pkg.add("BenchmarkTools")
      using BenchmarkTools
```

```
   Resolving package versions...
   Installed JSON ──────── v1.11.0
   Installed StructUtils ─ v2.10.0
   Installed Parsers ───── v2.8.8
    Updating `~/Project.toml`
  [6e4b80f9] + BenchmarkTools v1.8.0
    Updating `~/Manifest.toml`
  [6e4b80f9] + BenchmarkTools v1.8.0
  [682c06a0] + JSON v1.11.0
^ [69de0a69] + Parsers v2.8.8
  [ec057cc2] + StructUtils v2.10.0
  [9abbd945] + Profile
        Info Packages marked with ^ have new versions available and may be upgradable.
Precompiling project...
  ✓ StructUtils
  ✓ StructUtils → StructUtilsTablesExt
  ✓ Parsers
  ✓ InlineStrings → ParsersExt
  ✓ StructUtils → StructUtilsDataFramesExt
  ✓ JSON
  ✓ BenchmarkTools
  7 dependencies successfully precompiled in 25 seconds. 56 already precompiled.
```

```
[63]: n = 1000
      A = randn(n, n)
```

```
[63]: 1000×1000 Matrix{Float64}:
       -0.460132   -0.257948   -0.48148    …   0.422691    -0.48718    -0.251793
       -0.31444    -0.122083   -0.451102      -0.589617    -0.252339   -0.780525
        0.0269166   1.5563      0.632321       0.00218506  -0.546557   -1.4287
        0.675658   -1.01407     2.36924        0.590106    -0.0219674  -0.0989571
       -2.23539    -0.532248   -2.14269        0.75607     -0.022834    0.365478
        1.36642    -0.568713   -0.699825   …  -0.406969    -0.43223     0.370275
        0.221506   -0.466937   -0.791265       0.86386     -0.80765     0.227438
       -1.3118      0.506535   -0.512496       0.887233     0.700548   -0.591787
        1.12395     1.59715     0.197924      -0.41094     -0.244689    0.150454
        0.561705   -0.357374   -0.958702      -0.596862    -1.86262     1.19807
       -1.09653     0.26504    -1.26083    …  -1.26717     -0.648896   -1.38684
       -1.2999     -1.13183    -1.11401       -0.493481    -1.36817     0.294094
```

Рисунок 4.12: Проверка спектрального разложения, подключение BenchmarkTools, матрица 1000×1000

```
[64]: Asym = A + A'
```

```
[64]: 1000×1000 Matrix{Float64}:
       -0.920264  -0.572387   -0.454563    …   2.28061    -2.30212    -0.552561
       -0.572387  -0.244167    1.1052          0.180757    1.20734     1.97537
       -0.454563   1.1052      1.26464        -0.787206    0.570851   -1.20503
        0.284915  -0.647396    2.55785         1.51369     1.42242    -0.068223
       -1.57087    0.0440629  -3.73321        -0.711211   -2.59348     0.0357818
        3.22063   -0.883926    1.12043    …   0.885379    -1.02031    -0.543958
       -0.249801  -1.53147    -0.111716        1.58648    -0.417563   -0.945468
       -0.848476   1.10799     0.403611        0.653543    0.364425   -1.20936
        0.284037   0.830992    0.00211881      0.218313    2.76019     1.39629
        0.966816  -2.59873    -1.26999        -0.240957   -2.49465     1.05677
       -1.95519   -0.285564   -0.741881    …  -0.684814   -1.4063     -1.47698
       -1.95412    0.1565      0.0717051      -1.45345    -0.340309    1.02328
       -1.97125    0.785259    2.49828         0.662784    2.01831     1.32203
        ⋮                                   ⋱
        1.86625    2.50377     0.738949        1.58622    -2.39723     1.81131
       -0.255935   1.08738     0.54863        -0.565528   -0.48986     0.0434501
       -0.910274   0.216268    1.09155    …   0.148943    -0.681788    0.0428284
       -0.853683  -0.444521   -0.923881       -3.29073     0.137223   -0.403883
        0.687184  -0.342964   -0.117319        2.06224     0.278201    0.899968
       -0.544346   2.59839     0.824841        0.876412   -1.15989     2.31909
       -0.554211  -1.0744     -2.9464          0.0421562  -0.222239    0.412695
       -1.16651    1.20946     1.47447    …   0.905916    -2.51772    -0.0238114
        0.137399  -0.419045   -1.80285        -0.498654   -2.24033     4.16324
        2.28061    0.180757   -0.787206        0.325278    0.703664   -0.970111
       -2.30212    1.20734     0.570851        0.703664   -0.127345    0.0304313
       -0.552561   1.97537    -1.20503        -0.970111    0.0304313  -1.83156
```

Рисунок 4.13: Симметризованная матрица 1000×1000

Проверка `issymmetric` для симметризованной матрицы даёт `true`. После добавления шума `5eps()` в элемент `[1, 2]` матрица перестаёт быть симметричной (`false`), для неё явно задана структура `Symmetric` (рис. -Рисунок 4.14).

```
[65]: issymmetric(Asym)

[65]: true

[66]: Asym_noisy = copy(Asym)
      Asym_noisy[1, 2] += 5eps()

[66]: -0.5723874623437681

[67]: issymmetric(Asym_noisy)

[67]: false

[68]: Asym_explicit = Symmetric(Asym_noisy)

[68]: 1000×1000 Symmetric{Float64, Matrix{Float64}}:
 -0.920264  -0.572387   -0.454563    …   2.28061    -2.30212    -0.552561
 -0.572387  -0.244167    1.1052          0.180757    1.20734     1.97537
 -0.454563   1.1052      1.26464        -0.787206    0.570851   -1.20503
  0.284915  -0.647396    2.55785         1.51369     1.42242    -0.068223
 -1.57087    0.0440629  -3.73321        -0.711211   -2.59348     0.0357818
  3.22063   -0.883926    1.12043    …   0.885379   -1.02031    -0.543958
 -0.249801  -1.53147    -0.111716        1.58648    -0.417563   -0.945468
 -0.848476   1.10799     0.403611        0.653543    0.364425   -1.20936
  0.284037   0.830992    0.00211881      0.218313    2.76019     1.39629
  0.966816  -2.59873    -1.26999        -0.240957   -2.49465     1.05677
 -1.95519   -0.285564   -0.741881    …  -0.684814   -1.4063     -1.47698
 -1.95412    0.1565      0.0717051      -1.45345    -0.340309    1.02328
 -1.97125    0.785259    2.49828         0.662784    2.01831     1.32203
  ⋮                                   ⋱
  1.86625    2.50377     0.738949        1.58622    -2.39723     1.81131
 -0.255935   1.08738     0.54863        -0.565528   -0.48986     0.0434501
 -0.910274   0.216268    1.09155    …   0.148943   -0.681788    0.0428284
 -0.853683  -0.444521   -0.923881       -3.29073     0.137223   -0.403883
  0.687184  -0.342964   -0.117319        2.06224     0.278201    0.899968
 -0.544346   2.59839     0.824841        0.876412   -1.15989     2.31909
 -0.554211  -1.0744     -2.9464          0.0421562  -0.222239    0.412695
 -1.16651    1.20946     1.47447    …   0.905916   -2.51772    -0.0238114
  0.137399  -0.419045   -1.80285        -0.498654   -2.24033     4.16324
  2.28061    0.180757   -0.787206        0.325278    0.703664   -0.970111
 -2.30212    1.20734     0.570851        0.703664   -0.127345    0.0304313
 -0.552561   1.97537    -1.20503        -0.970111    0.0304313  -1.83156
```

Рисунок 4.14: Проверка симметричности, добавление шума, явное указание структуры Symmetric

Оценено время вычисления собственных значений для трёх матриц (рис. -Рисунок 4.15).

```
[69]: @btime eigvals(Asym)
        58.842 ms (11 allocations: 7.99 MiB)
[69]: 1000-element Vector{Float64}:
       -88.5880882895207
       -87.22403563902796
       -86.53970506681245
       -85.9228629442112
       -85.52716571067768
       -85.41863569980583
       -85.28887410886152
       -84.87220392781805
       -84.5744580368861
       -83.57280114653356
       -82.58423529712789
       -82.47423981510988
       -82.31187879143523
         ⋮
        82.52834805521893
        82.96161984665909
        83.67251148275285
        84.09522840363472
        84.24574603845429
        84.88485355627193
        85.17015932492703
        85.59233703735127
        86.06210054434527
        86.28033094255817
        87.67937081712803
        88.54152515487337
[70]: @btime eigvals(Asym_noisy);
        597.514 ms (14 allocations: 7.93 MiB)
[71]: @btime eigvals(Asym_explicit);
        57.799 ms (11 allocations: 7.99 MiB)
```

Рисунок 4.15: Время вычисления собственных значений: симметричная, зашумлённая и явно симметричная матрицы

Для разреженной структуры создана трёхдиагональная симметричная матрица `SymTridiagonal` размера $10^6 \times 10^6$ и измерено время нахождения наибольшего собственного значения `eigmax` (рис. -Рисунок 4.16).

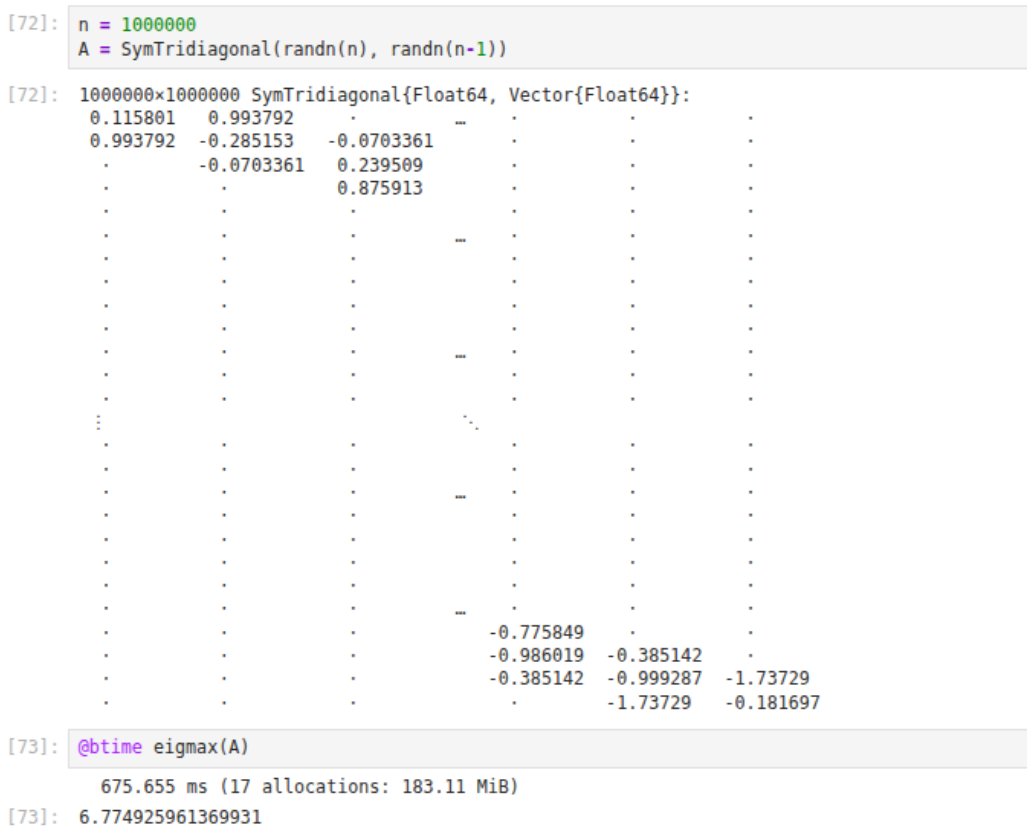


Рисунок 4.16: Трёхдиагональная матрица $10^6 \times 10^6$ и время вычисления eigmax

4.7 Общая линейная алгебра (раздел 4.2.6)

Создана матрица с рациональными элементами `Rational{BigInt}` и вектор `b = Arational * x`. Решение СЛАУ оператором `\` вернуло вектор из единиц в точном (рациональном) виде, без погрешностей округления; выполнено LU-разложение рациональной матрицы (рис. -Рисунок 4.17).

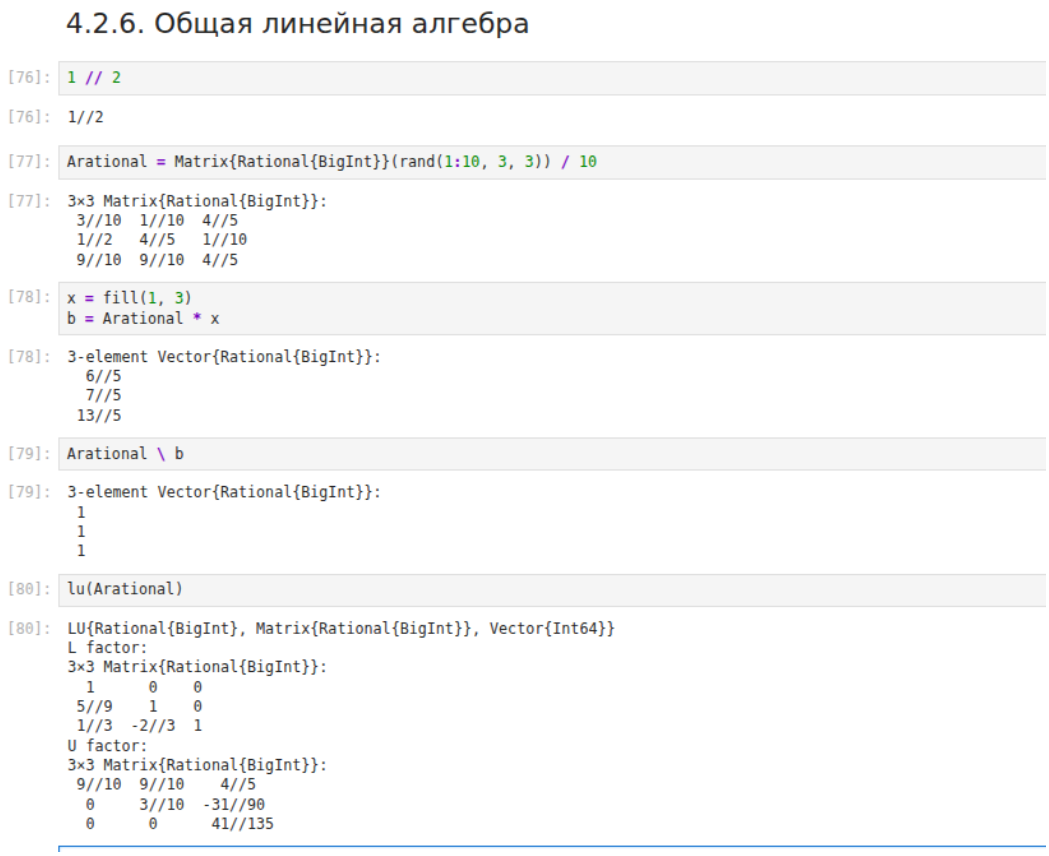


Рисунок 4.17: Решение СЛАУ и LU-разложение для матрицы с рациональными элементами

4.8 Задания для самостоятельной работы (раздел 4.4)

4.8.1 Произведение векторов (4.4.1)

Задан вектор `v = [1, 2, 3]`. Скалярное произведение `v` на себя сохранено в `dot_v` (получено 14), внешнее произведение `v * v'` сохранено в `outer_v` (матрица 3×3) (рис. -Рисунок 4.18).

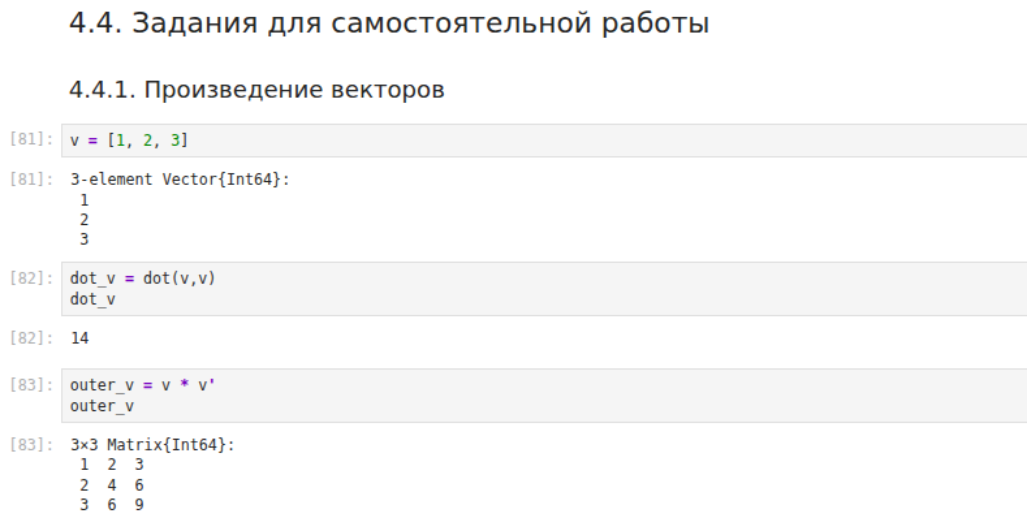


Рисунок 4.18: Скалярное и внешнее произведение вектора на себя

4.8.2 Системы линейных уравнений (4.4.2)

Для анализа систем $Ax = b$ написана функция `solve_slau`, которая сравнивает ранг матрицы системы $\text{rank}(A)$ и ранг расширенной матрицы $\text{rank}([A|b])$ (теорема Кронекера — Капелли), определяет тип системы (единственное решение, бесконечно много решений, несовместность) и пробует решить систему оператором $\backslash$ (рис. -Рисунок 4.19).

Системы с двумя неизвестными. Система (a) имеет единственное решение $x = 2.5$, $y = -0.5$; в системе (b) уравнения пропорциональны (ранги равны 1, решений бесконечно много), оператор $\backslash$ выдаёт `SingularException`; система (c) несовместна ($\text{rank}(A) = 1 < \text{rank}([A|b]) = 2$).

4.4.2. Системы линейных уравнений

[84]:
```
function solve_slau(A, b; label="")
    ra = rank(A)                # ранг матрицы системы
    rab = rank(hcat(A, b))      # ранг расширенной матрицы
    n = size(A, 2)              # число неизвестных
    println(label, " rank(A) = $ra, rank([A|b]) = $rab, неизвестных: $n")
    if ra < rab
        println("   система несовместна, решений нет")
    elseif ra == n
        println("   единственное решение")
    else
        println("   бесконечно много решений")
    end
    try
        println("   A\\b = ", A \ b)
    catch e
        println("   A\\b не вычисляется: ", typeof(e))
    end
end
```

[84]:
```
solve_slau (generic function with 1 method)
```

[85]:
```
#1a) x + y = 2, x - y = 3
solve_slau([1 1; 1 -1], [2, 3]; label="a)")
```

```
a) rank(A) = 2, rank([A|b]) = 2, неизвестных: 2
   единственное решение
   A\b = [2.5, -0.5]
```

[86]:
```
# 1b) x + y = 2, 2x + 2y = 4
solve_slau([1 1; 2 2], [2, 4]; label="b)")
```

```
b) rank(A) = 1, rank([A|b]) = 1, неизвестных: 2
   бесконечно много решений
   A\b не вычисляется: SingularException
```

[87]:
```
# 1c) x + y = 2, 2x + 2y = 5
solve_slau([1 1; 2 2], [2. 5]; label="c)")
```

```
DimensionMismatch: number of rows of each array must match (got (2, 1))

Stacktrace:
 [1] _typed_hcat(::Type{Float64}, A::Tuple{Matrix{Int64}, Matrix{Float64}})
   @ Base ./abstractarray.jl:1667
 [2] typed_hcat
   @ ./abstractarray.jl:1646 [inlined]
 [3] hcat
   @ /opt/julia/share/julia/stdlib/v1.10/SparseArrays/src/sparsevector.jl:1233 [inlined]
 [4] solve_slau(A::Matrix{Int64}, b::Matrix{Float64}; label::String)
   @ Main ./In[84]:3
 [5] top-level scope
   @ In[87]:2
```

Рисунок 4.19: Функция solve_slau и системы с двумя неизвестными (a)–(c)

В системе (d) три пропорциональных уравнения, решений бесконечно много, оператор \ возвращает решение минимальной нормы. Система (e) несовместна, оператор \ возвращает приближённое решение по методу наименьших квадратов. Система (f) совместна и имеет единственное решение $x = -1$, $y = 3$ (рис. -Рисунок 4.20).

```
[88]: # 1d) x + y = 1, 2x + 2y = 2, 3x + 3y = 3
solve_slau([1 1; 2 2; 3 3], [1, 2, 3]; label="d)")

d) rank(A) = 1, rank([A|b]) = 1, неизвестных: 2
   бесконечно много решений
   A\b = [0.4999999999999999, 0.5]

[89]: # 1e) x + y = 2, 2x + y = 1, x - y = 3
solve_slau([1 1; 2 1; 1 -1], [2, 1, 3]; label="e)")

e) rank(A) = 2, rank([A|b]) = 3, неизвестных: 2
   система несовместна, решений нет
   A\b = [1.5000000000000004, -0.9999999999999997]

[90]: # 1f) x + y = 2, 2x + y = 1, 3x + 2y = 3
solve_slau([1 1; 2 1; 3 2], [2, 1, 3]; label="f)")

f) rank(A) = 2, rank([A|b]) = 2, неизвестных: 2
   единственное решение
   A\b = [-0.9999999999999989, 2.9999999999999982]
```

Рисунок 4.20: Системы с двумя неизвестными (d)–(f)

Системы с тремя неизвестными (рис. -Рисунок 4.21): в (a) два уравнения и три неизвестных — решений бесконечно много; (b) — единственное решение $(-0.5;\ 2.5;\ 0)$; (c) — решений бесконечно много (матрица вырождена, ранг 2, система совместна); (d) — система несовместна ($\text{rank}(A) = 2$, $\text{rank}([A|b]) = 3$).

```
[91]: # 2a) x + y + z = 2, x - y - 2z = 3
solve_slau([1 1 1; 1 -1 -2], [2, 3]; label="a)")

a) rank(A) = 2, rank([A|b]) = 2, неизвестных: 3
   бесконечно много решений
   A\b = [2.2142857142857144, 0.35714285714285704, -0.5714285714285712]

[92]: # 2b) x + y + z = 2, 2x + 2y - 3z = 4, 3x + y + z = 1
solve_slau([1 1 1; 2 2 -3; 3 1 1], [2, 4, 1]; label="b)")

b) rank(A) = 3, rank([A|b]) = 3, неизвестных: 3
   единственное решение
   A\b = [-0.5, 2.5, 0.0]

[93]: # 2c) x + y + z = 1, x + y + 2z = 0, 2x + 2y + 3z = 1
solve_slau([1 1 1; 1 1 2; 2 2 3], [1, 0, 1]; label="c)")

c) rank(A) = 2, rank([A|b]) = 2, неизвестных: 3
   бесконечно много решений
   A\b не вычисляется: SingularException

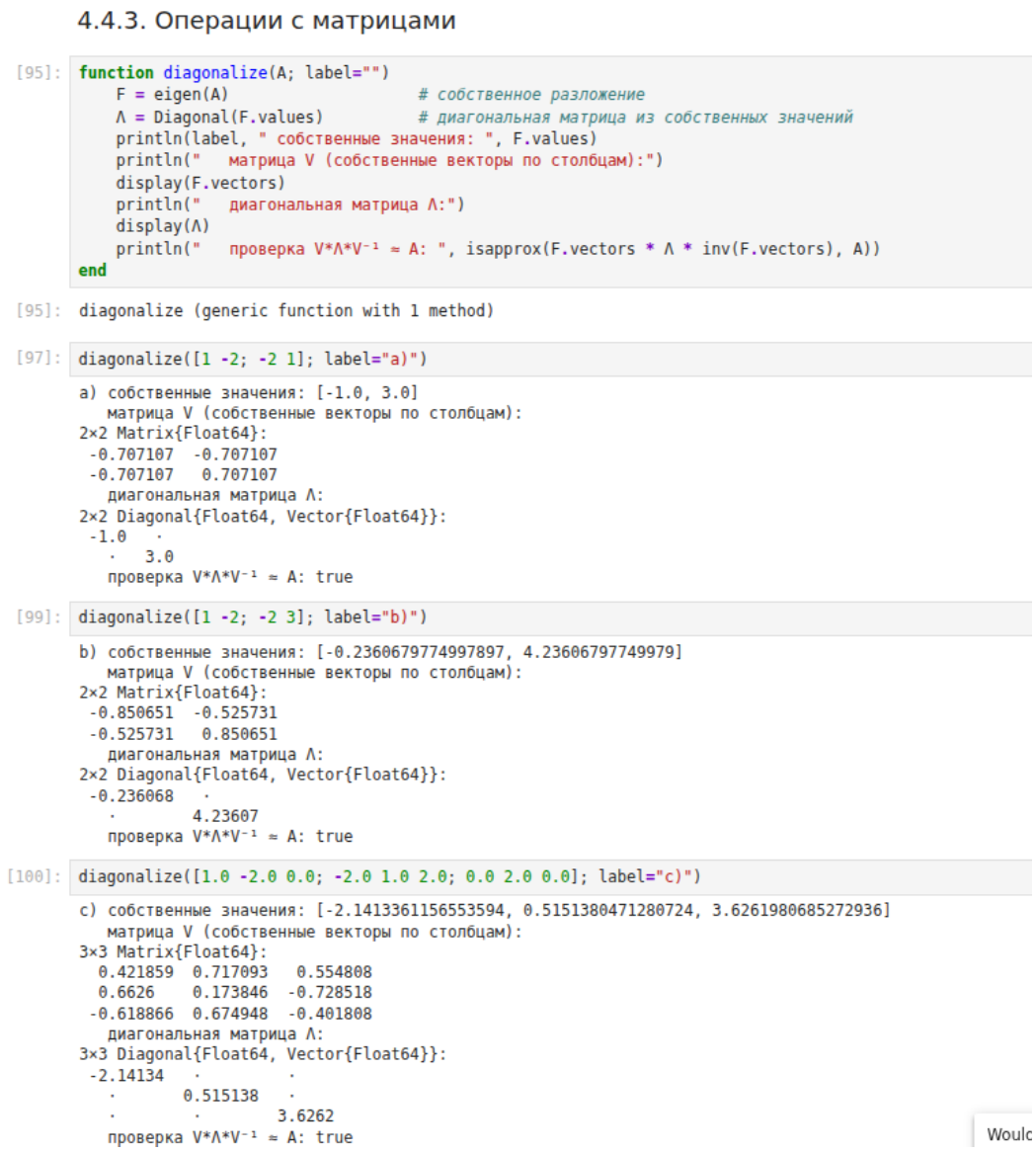
[94]: # 2d) x + y + z = 1, x + y + 2z = 0, 2x + 2y + 3z = 0
solve_slau([1 1 1; 1 1 2; 2 2 3], [1, 0, 0]; label="d)")

d) rank(A) = 2, rank([A|b]) = 3, неизвестных: 3
   система несовместна, решений нет
   A\b не вычисляется: SingularException
```

Рисунок 4.21: Системы с тремя неизвестными (a)–(d)

4.8.3 Операции с матрицами (4.4.3)

Написана функция `diagonalize`, выполняющая спектральное разложение $A = V\Lambda V^{-1}$ и проверку результата. Для матриц (a)–(c) получены собственные значения: (a) -1 и 3; (b) -0.236 и 4.236; (c) -2.141, 0.515, 3.626. Во всех случаях проверка $V\Lambda V^{-1} \approx A$ дала `true` (рис. -Рисунок 4.22).



4.4.3. Операции с матрицами

[95]:
```
function diagonalize(A; label="")
    F = eigen(A)                    # собственное разложение
    Λ = Diagonal(F.values)          # диагональная матрица из собственных значений
    println(label, " собственные значения: ", F.values)
    println("   матрица V (собственные векторы по столбцам):")
    display(F.vectors)
    println("   диагональная матрица Λ:")
    display(Λ)
    println("   проверка V*Λ*V⁻¹ ≈ A: ", isapprox(F.vectors * Λ * inv(F.vectors), A))
end
```

[95]:
```
diagonalize (generic function with 1 method)
```

[97]:
```
diagonalize([1 -2; -2 1]; label="a)")
```

```
a) собственные значения: [-1.0, 3.0]
   матрица V (собственные векторы по столбцам):
2×2 Matrix{Float64}:
 -0.707107  -0.707107
 -0.707107   0.707107
   диагональная матрица Λ:
2×2 Diagonal{Float64, Vector{Float64}}:
 -1.0   ⋅
   ⋅   3.0
   проверка V*Λ*V⁻¹ ≈ A: true
```

[99]:
```
diagonalize([1 -2; -2 3]; label="b)")
```

```
b) собственные значения: [-0.2360679774997897, 4.23606797749979]
   матрица V (собственные векторы по столбцам):
2×2 Matrix{Float64}:
 -0.850651  -0.525731
 -0.525731   0.850651
   диагональная матрица Λ:
2×2 Diagonal{Float64, Vector{Float64}}:
 -0.236068   ⋅
   ⋅        4.23607
   проверка V*Λ*V⁻¹ ≈ A: true
```

[100]:
```
diagonalize([1.0 -2.0 0.0; -2.0 1.0 2.0; 0.0 2.0 0.0]; label="c)")
```

```
c) собственные значения: [-2.1413361156553594, 0.5151380471280724, 3.6261980685272936]
   матрица V (собственные векторы по столбцам):
3×3 Matrix{Float64}:
  0.421859  0.717093   0.554808
  0.6626    0.173846  -0.728518
 -0.618866  0.674948  -0.401808
   диагональная матрица Λ:
3×3 Diagonal{Float64, Vector{Float64}}:
 -2.14134   ⋅         ⋅
   ⋅       0.515138   ⋅
   ⋅        ⋅        3.6262
   проверка V*Λ*V⁻¹ ≈ A: true
```

Woulc

Рисунок 4.22: Приведение матриц к диагональному виду

Вычислены функции от матриц (рис. -Рисунок 4.23):

- $\begin{pmatrix} 1 & -2 \\ -2 & 1 \end{pmatrix}^{10} = \begin{pmatrix} 29525 & -29524 \\ -29524 & 29525 \end{pmatrix}$;
- квадратный корень из $\begin{pmatrix} 5 & -2 \\ -2 & 5 \end{pmatrix}$ — встроенной функцией `sqrt`, проверка: квадрат результата равен исходной матрице;
- кубический корень из $\begin{pmatrix} 1 & -2 \\ -2 & 1 \end{pmatrix}$ — через собственное разложение $V \cdot \mathrm{diag}(\sqrt[3]{\lambda_i}) \cdot V^{-1}$, проверка: куб результата равен исходной матрице.

2. Вычисление со степенями и корнями матриц

```
[101]: # a) Матрица в десятой степени
       A_a = [1 -2; -2 1]
       A_a^10
```

```
[101]: 2×2 Matrix{Int64}:
         29525  -29524
        -29524   29525
```

```
[102]: # b) Квадратный корень из матрицы
       A_b = [5 -2; -2 5]
       sqrt(A_b)
```

```
[102]: 2×2 Matrix{Float64}:
         2.1889   -0.45685
        -0.45685   2.1889
```

```
[103]: sqrt(A_b)^2
```

```
[103]: 2×2 Matrix{Float64}:
         5.0  -2.0
        -2.0   5.0
```

```
[104]: # c) Кубический корень через разложение
       A_c = [1 -2; -2 1]
       F = eigen(A_c)
       cbrt_A = F.vectors * Diagonal(cbrt.(F.values)) * inv(F.vectors)
```

```
[104]: 2×2 Matrix{Float64}:
         0.221125  -1.22112
        -1.22112    0.221125
```

```
[105]: cbrt_A^3
```

```
[105]: 2×2 Matrix{Float64}:
         1.0  -2.0
        -2.0   1.0
```

Рисунок 4.23: Степень, квадратный и кубический корни матриц

У матрицы $\begin{pmatrix} 1 & 2 \\ 2 & 3 \end{pmatrix}$ одно собственное значение отрицательное (-0.236 и 4.236), поэтому вещественного квадратного корня не существует. Функция `sqrt` вернула комплексную матрицу, тот же результат получен через собственное разложение с переходом к комплексным числам; проверка (квадрат результата) вернула исходную матрицу с точностью до мнимых частей порядка 10^{-17} (рис. -Рисунок 4.24).

```
[106]: # d) матрица и её собственные значения
       A_d = [1 2; 2 3]
       eigvals(A_d)

[106]: 2-element Vector{Float64}:
        -0.2360679774997897
         4.23606797749979

[107]: try
           display(sqrt(A_d))
       catch e
           println("sqrt(A_d) не вычисляется: ", typeof(e))
       end

       2×2 Matrix{ComplexF64}:
        0.568864+0.351578im  0.920442-0.217287im
        0.920442-0.217287im   1.48931+0.134291im

[108]: F_d = eigen(A_d)
       sqrt_A_d = F_d.vectors * Diagonal(sqrt.(Complex.(F_d.values))) * inv(F_d.vectors)

[108]: 2×2 Matrix{ComplexF64}:
        0.568864+0.351578im  0.920442-0.217287im
        0.920442-0.217287im   1.48931+0.134291im

[109]: sqrt_A_d^2

[109]: 2×2 Matrix{ComplexF64}:
        1.0-5.55112e-17im  2.0+5.55112e-17im
        2.0+0.0im          3.0+5.55112e-17im
```

Рисунок 4.24: Квадратный корень из матрицы с отрицательным собственным значением

Для симметричной матрицы 5×5 вычислены собственные значения (-128.493; -55.888; 42.752; 87.161; 542.468), построены диагональная матрица из собственных значений (`Diagonal`) и нижнетреугольная матрица (`LowerTriangular`), измерено время выполнения операций макросом `@btime` (рис. -Рисунок 4.25).

```
[110]: # Матрица A 5x5
       A5 = [140  97  74 168 131;
              97 106  89 131  36;
              74  89 152 144  71;
             168 131 144  54 142;
             131  36  71 142  36]
```

```
[110]: 5×5 Matrix{Int64}:
        140   97   74  168  131
         97  106   89  131   36
         74   89  152  144   71
        168  131  144   54  142
        131   36   71  142   36
```

```
[111]: eigvals(A5)
```

```
[111]: 5-element Vector{Float64}:
       -128.49322764802145
        -55.88778455305688
         42.75216727931894
         87.16111477514521
        542.4677301466143
```

```
[112]: A5 = Diagonal(eigvals(A5))
```

```
[112]: 5×5 Diagonal{Float64, Vector{Float64}}:
       -128.493      ⋅         ⋅         ⋅         ⋅
          ⋅      -55.8878     ⋅         ⋅         ⋅
          ⋅         ⋅      42.7522      ⋅         ⋅
          ⋅         ⋅         ⋅      87.1611      ⋅
          ⋅         ⋅         ⋅         ⋅      542.468
```

```
[113]: L5 = LowerTriangular(A5)
```

```
[113]: 5×5 LowerTriangular{Float64, Diagonal{Float64, Vector{Float64}}}:
       -128.493      ⋅         ⋅         ⋅         ⋅
          0.0     -55.8878     ⋅         ⋅         ⋅
          0.0       0.0     42.7522      ⋅         ⋅
          0.0       0.0       0.0     87.1611      ⋅
          0.0       0.0       0.0      0.0     542.468
```

```
[114]: @btime eigvals($A5);
```

```
  24.105 ns (1 allocation: 96 bytes)
```

```
[115]: @btime Diagonal(eigvals($A5));
```

```
  24.086 ns (1 allocation: 96 bytes)
```

```
[116]: @btime LowerTriangular($A5);
```

```
  2.709 ns (0 allocations: 0 bytes)
```

Рисунок 4.25: Собственные значения матрицы 5×5, диагональная и нижнетреугольная матрицы, оценка времени

4.8.4 Линейные модели экономики (4.4.4)

Продуктивность матриц проверена тремя способами. По определению: для базисных неотрицательных векторов $y = (1, 0)$ и $y = (0, 1)$ решается система $(E - A)x = y$ и проверяется неотрицательность x (рис. -Рисунок 4.26). Матрицы (a) и (b) не продуктивны (в решении появляются отрицательные компоненты), матрица (c)

продуктивна.

4.4.4. Линейные модели экономики

1. Проверка продуктивности по определению

```
17]: function check_productive_def(A; label="")
         E = Matrix{Float64}(I, 2, 2)       # единичная матрица
         println(label, " A = ", A)
         for y in ([1.0, 0.0], [0.0, 1.0])
             x = (E - A) \ y                # решение системы (E - A)x = y
             println("   y = $y -> x = $x, все x ≥ 0: ", all(x .>= 0))
         end
     end

17]: check_productive_def (generic function with 1 method)

18]: # a) A = [1 2; 3 4]
     check_productive_def([1 2; 3 4]; label="a)")

     a) A = [1 2; 3 4]
        y = [1.0, 0.0] -> x = [0.5, -0.5], все x ≥ 0: false
        y = [0.0, 1.0] -> x = [-0.3333333333333333, -0.0], все x ≥ 0: false

19]: # b) A = 1/2 * [1 2; 3 4]
     check_productive_def([1 2; 3 4] / 2; label="b)")

     b) A = [0.5 1.0; 1.5 2.0]
        y = [1.0, 0.0] -> x = [0.5, -0.75], все x ≥ 0: false
        y = [0.0, 1.0] -> x = [-0.5, -0.25], все x ≥ 0: false

20]: # c) A = 1/10 * [1 2; 3 4]
     check_productive_def([1 2; 3 4] / 10; label="c)")

     c) A = [0.1 0.2; 0.3 0.4]
        y = [1.0, 0.0] -> x = [1.25, 0.625], все x ≥ 0: true
        y = [0.0, 1.0] -> x = [0.41666666666666663, 1.875], все x ≥ 0: true
```

Рисунок 4.26: Проверка продуктивности по определению

По критерию $(E - A)^{-1} \geq 0$ (рис. -Рисунок 4.27): для матрицы (a) обратная матрица содержит отрицательные элементы, для (b) — тоже, для (c) все элементы неотрицательны.

2. Критерий продуктивности через (E - A)^-1

[121]:
```
function check_productive_inv(A; label="")
    E = Matrix{Float64}(I, size(A, 1), size(A, 1))
    M = inv(E - A)                        # обратная матрица (E - A)⁻¹
    println(label, " (E-A)⁻¹ = ")
    display(M)
    println("   все элементы ≥ 0: ", all(M .>= 0))
end
```

[121]:
```
check_productive_inv (generic function with 1 method)
```

[122]:
```
# a) A = [1 2; 3 1]
check_productive_inv([1 2; 3 1]; label="a)")
```

```
a) (E-A)⁻¹ =
2×2 Matrix{Float64}:
 -0.0  -0.333333
 -0.5   0.0
   все элементы ≥ 0: false
```

[123]:
```
# b) A = 1/2 * [1 2; 3 1]
check_productive_inv([1 2; 3 1] / 2; label="b)")
```

```
b) (E-A)⁻¹ =
2×2 Matrix{Float64}:
 -0.4  -0.8
 -1.2  -0.4
   все элементы ≥ 0: false
```

[124]:
```
# c) A = 1/10 * [1 2; 3 1]
check_productive_inv([1 2; 3 1] / 10; label="c)")
```

```
c) (E-A)⁻¹ =
2×2 Matrix{Float64}:
 1.2  0.266667
 0.4  1.2
   все элементы ≥ 0: true
```

Рисунок 4.27: Критерий продуктивности через матрицу $(E - A)^{-1}$

По спектральному критерию (рис. -Рисунок 4.28): у матрицы (a) собственные значения -1.449 и 3.449, у (b) — -0.725 и 1.725 (есть $|\lambda| > 1$, матрицы не продуктивны); у (c) — -0.145 и 0.345, у (d) — 0.027; 0.1; 0.373 (все $|\lambda| < 1$, матрицы продуктивны).

3. Спектральный критерий продуктивности

[125]:
```
function check_productive_spec(A; label="")
    λ = eigvals(A)                        # собственные значения
    println(label, " собственные значения: ", λ)
    println("   все |λ| < 1: ", all(abs.(λ) .< 1))
end
```

[125]:
```
check_productive_spec (generic function with 1 method)
```

[126]:
```
check_productive_spec([1 2; 3 1]; label="a)")
```

```
a) собственные значения: [-1.4494897427831779, 3.4494897427831783]
   все |λ| < 1: false
```

[127]:
```
check_productive_spec([1 2; 3 1] / 2; label="b)")
```

```
b) собственные значения: [-0.7247448713915892, 1.724744871391589]
   все |λ| < 1: false
```

[128]:
```
check_productive_spec([1 2; 3 1] / 10; label="c)")
```

```
c) собственные значения: [-0.14494897427831785, 0.34494897427831783]
   все |λ| < 1: true
```

[129]:
```
check_productive_spec([0.1 0.2 0.3; 0 0.1 0.2; 0 0.1 0.3]; label="d)")
```

```
d) собственные значения: [0.02679491924311228, 0.1, 0.37320508075688774]
   все |λ| < 1: true
```

Рисунок 4.28: Спектральный критерий продуктивности

5 Результаты

- Примеры раздела 4.2 воспроизведены, результаты соответствуют методическим указаниям: суммы, произведения и средние по осям, транспонирование, след, ранг, определитель, обратная и псевдообратная матрицы, нормы и расстояния, повороты, матричное и скалярное произведения, LU-, QR- и спектральное разложения, работа с матрицами специальной структуры и с рациональными матрицами.
- Скалярное произведение векторов $X = (2, 4, -5)$ и $Y = (1, -1, 3)$ равно -17 при вычислении как `dot(X, Y)`, так и `X' * Y`.
- Время вычисления собственных значений матрицы 1000×1000 (рис. -Рисунок 4.15): симметричная матрица — 58.8 мс, зашумлённая (несимметричная) — 597.5 мс, зашумлённая с явным указанием `Symmetric` — 57.8 мс.
- Наибольшее собственное значение трёхдиагональной матрицы $10^6 \times 10^6$ типа `SymTridiagonal` найдено за 675.7 мс при выделении 183.11 МиБ памяти.
- Система с рациональной матрицей решена точно: получен вектор $(1, 1, 1)$ без погрешностей округления.
- Задание 4.4.1: dot_v $= 14$, `outer_v` — матрица 3×3 с элементами $v_i v_j$.
- Задание 4.4.2: определены типы всех десяти систем (табл. -Таблица 5.1).
- Задание 4.4.3: матрицы приведены к диагональному виду, вычислены степень, квадратные и кубический корни, найдены собственные значения матрицы 5×5.

- Задание 4.4.4: матрицы (a) и (b) не продуктивны, матрица (c) продуктивна по всем применённым критериям; матрица (d) продуктивна по спектральному критерию.

Таблица 5.1: Результаты решения систем линейных уравнений

Система	Число неизвестных	Тип системы
1a	2	единственное решение $(2.5;\ -0.5)$
1b	2	бесконечно много решений
1c	2	несовместна
1d	2	бесконечно много решений
1e	2	несовместна
1f	2	единственное решение $(-1;\ 3)$
2a	3	бесконечно много решений
2b	3	единственное решение $(-0.5;\ 2.5;\ 0)$
2c	3	бесконечно много решений
2d	3	несовместна

6 Обсуждение результатов

Структура матрицы и скорость вычислений. Явное указание структуры заметно влияет на время вычислений. Зашумлённая матрица отличается от симметричной на величину порядка машинной точности в одном элементе, но Julia не распознаёт её как симметричную (`issymmetric` возвращает `false`) и использует общий алгоритм, который примерно в 10 раз медленнее (597.5 мс против 58.8 мс). Обёртка `Symmetric` позволяет вернуться к специализированному алгоритму, и время вычисления (57.8 мс) совпадает со временем для точно симметричной матрицы. Для трёхдиагональной матрицы $10^6 \times 10^6$ тип `SymTridiagonal` хранит только две диагонали (183 МиБ, включая временные данные вычислений), тогда как плотная матрица такого размера потребовала бы около $10^{12} \cdot 8$ байт, то есть порядка 8 ТБ, и вызвала бы ошибку нехватки памяти (`OutOfMemoryError`), на что указывают методические указания.

Решение СЛАУ. Оператор `\` ведёт себя по-разному в зависимости от формы матрицы: для невырожденной квадратной матрицы возвращает точное решение, для вырожденной квадратной — выбрасывает `SingularException`, для прямоугольной — возвращает решение методом наименьших квадратов (для системы с бесконечным числом решений — решение минимальной нормы). Поэтому тип системы нужно определять независимо — сравнением рангов, как реализовано в функции `solve_slau`: значение `A\b` для несовместной системы (1e) является лишь приближением и не удовлетворяет всем уравнениям.

Корни матриц. Квадратный корень существует и вещественен, если все соб-

ственные значения неотрицательны. Для матрицы $\begin{pmatrix} 1 & 2 \\ 2 & 3 \end{pmatrix}$ это не так, поэтому корень получается комплексным. Кубический корень вещественной матрицы существует и при отрицательных собственных значениях, так как кубический корень из отрицательного вещественного числа вещественен.

Критерии продуктивности. Все три критерия для матриц (a)–(c) дали согласованный результат, что подтверждает их эквивалентность: матрица продуктивна тогда и только тогда, когда спектральный радиус меньше 1, что равносильно неотрицательности $(E - A)^{-1}$. Уменьшение коэффициентов матрицы (умножение на $1/2$, затем на $1/10$) уменьшает спектральный радиус и в итоге делает матрицу продуктивной.

Эффективность операций над матрицей 5×5**.** Операция `LowerTriangular(A)` является самой быстрой: она не копирует данные, а лишь оборачивает исходную матрицу в тип, указывающий на структуру, поэтому время её выполнения минимально и не зависит от размера матрицы. Вычисление собственных значений требует реального вычислительного алгоритма и заметно дороже.

7 Заключение

- Поставленная цель работы достигнута: изучены возможности пакетов `Statistics`, `LinearAlgebra` и `BenchmarkTools` для выполнения и оценки эффективности операций линейной алгебры.
- Воспроизведены все примеры раздела 4.2 и выполнены все задания раздела 4.4.
- Показано, что выбор специализированных типов матриц (`Symmetric`, `SymTridiagonal`, `LowerTriangular`) позволяет значительно ускорить вычисления и сократить расход памяти.
- Получены навыки решения СЛАУ с анализом их совместности по рангам, применения LU-, QR- и спектрального разложений, вычисления функций от матриц и проверки продуктивности матриц в линейных моделях экономики.
- Рекомендация: при работе с матрицами большой размерности и известной структурой явно указывать структуру матрицы, а при решении СЛАУ предварительно анализировать совместность системы.

Список литературы

1. Кулябов Д. С., Королькова А. В., Геворкян М. Н. Компьютерный практикум на языке Julia. — Российский университет дружбы народов.
2. Julia Documentation. LinearAlgebra. — URL: https://docs.julialang.org/en/v1/stdlib/LinearAlgebra/.
3. BenchmarkTools.jl. — URL: https://github.com/JuliaCI/BenchmarkTools.jl.
4. Klok H., Nazarathy Y. Statistics with Julia: Fundamentals for Data Science, Machine Learning and Artificial Intelligence. — 2020. — URL: https://statisticswithjulia.org/.
5. Антонюк В. А. Язык Julia как инструмент исследователя. — М. : Физический факультет МГУ им. М. В. Ломоносова, 2019.

A Листинг кода блокнота linear-algebra.ipynb

```
# --- 4.2.1. [illegible] [illegible] [illegible] [illegible] [illegible] ---

a = rand(1:20, (4, 3))
sum(a), sum(a, dims=1), sum(a, dims=2)
prod(a), prod(a, dims=1), prod(a, dims=2)

import Pkg
Pkg.add("Statistics")
using Statistics
mean(a), mean(a, dims=1), mean(a, dims=2)

# --- 4.2.2. [illegible], [illegible], [illegible], [illegible], [illegible] ---

import Pkg
Pkg.add("LinearAlgebra")
using LinearAlgebra

b = rand(1:20, (4, 4))
transpose(b)
```

```
tr(b)
diag(b)
rank(b)
inv(b)
det(b)
pinv(b)

# --- 4.2.3. [illegible], [illegible] ---

X = [2, 4, -5]
norm(X)
p = 1
norm(X, p)

Y = [1, -1, 3]
norm(X - Y)
sqrt(sum((X - Y) .^ 2))
acos((transpose(X) * Y) / (norm(X) * norm(Y)))

d = [5 -4 2; -1 2 3; -2 1 0]
opnorm(d)
opnorm(d, 1)
rot180(d)
reverse(d, dims=1)
reverse(d, dims=2)

# --- 4.2.4. [illegible] [illegible], [illegible] [illegible], [illegible] [illegible] ---
```

```
A = rand(1:10, (2, 3))
B = rand(1:10, (3, 4))
A * B
Matrix{Int}(I, 3, 3)
dot(X, Y)
X' * Y

# --- 4.2.5. [illegible], [illegible] [illegible] [illegible] ---

A = rand(3, 3)
x = fill(1.0, 3)
b = A * x
A \ b

Alu = lu(A)
Alu.P
Alu.p
Alu.L
Alu.U
A \ b
Alu \ b
det(A)
det(Alu)

Aqr = qr(A)
Aqr.Q
Aqr.R
Aqr.Q' * Aqr.Q
```

```
Asym = A + A'
AsymEig = eigen(Asym)
AsymEig.values
AsymEig.vectors
inv(AsymEig) * Asym

import Pkg
Pkg.add("BenchmarkTools")
using BenchmarkTools

n = 1000
A = randn(n, n)
Asym = A + A'
issymmetric(Asym)

Asym_noisy = copy(Asym)
Asym_noisy[1, 2] += 5eps()
issymmetric(Asym_noisy)

Asym_explicit = Symmetric(Asym_noisy)

@btime eigvals(Asym);
@btime eigvals(Asym_noisy);
@btime eigvals(Asym_explicit);

n = 1000000
A = SymTridiagonal(randn(n), randn(n - 1))
@btime eigmax(A)
```

```
# --- 4.2.6. □□□□□ □□□□□□□□ □□□□□□□ ---

1 // 2
Arational = Matrix{Rational{BigInt}}(rand(1:10, 3, 3)) / 10
x = fill(1, 3)
b = Arational * x
Arational \ b
lu(Arational)

# --- 4.4.1. □□□□□□□□□□□□ □□□□□□□□ ---

v = [1, 2, 3]
dot_v = dot(v, v)
outer_v = v * v'

# --- 4.4.2. □□□□□□□ □□□□□□□□ □□□□□□□□□ ---

function solve_slau(A, b; label="")
    ra = rank(A)
    rab = rank(hcat(A, b))
    n = size(A, 2)
   println(label, " rank(A) = $ra, rank([A|b]) = $rab, □□□□□□□□□□□: $n")
    if ra < rab
        println("   □□□□□□□ □□□□□□□□□□□, □□□□□□□ □□□")
    elseif ra == n
        println("   □□□□□□□□□□□ □□□□□□□")
    else
        println("   □□□□□□□□□□ □□□□□ □□□□□□□")
```

```
    end
    try
        println("   A\\b = ", A \ b)
    catch e
        println("   A\\b □□ □□□□□□□□□□□□: ", typeof(e))
    end
end

solve_slau([1 1; 1 -1], [2, 3]; label="a)")
solve_slau([1 1; 2 2], [2, 4]; label="b)")
solve_slau([1 1; 2 2], [2, 5]; label="c)")
solve_slau([1 1; 2 2; 3 3], [1, 2, 3]; label="d)")
solve_slau([1 1; 2 1; 1 -1], [2, 1, 3]; label="e)")
solve_slau([1 1; 2 1; 3 2], [2, 1, 3]; label="f)")

solve_slau([1 1 1; 1 -1 -2], [2, 3]; label="a)")
solve_slau([1 1 1; 2 2 -3; 3 1 1], [2, 4, 1]; label="b)")
solve_slau([1 1 1; 1 1 2; 2 2 3], [1, 0, 1]; label="c)")
solve_slau([1 1 1; 1 1 2; 2 2 3], [1, 0, 0]; label="d)")

# --- 4.4.3. □□□□□□□□ □ □□□□□□□□□ ---

function diagonalize(A; label="")
    F = eigen(A)
    □ = Diagonal(F.values)
    println(label, " □□□□□□□□□□ □□□□□□□□: ", F.values)
    println("   □□□□□□□ V (□□□□□□□□□□□ □□□□□□□ □□ □□□□□□□□):")
    display(F.vectors)
```

```
    println("   □□□□□□□□□□□□ □□□□□□□ □:")
    display(□)
   println("  □□□□□□□□ V*□*V⁻¹ □ A: ", isapprox(F.vectors * □ * inv(F.vec
end

diagonalize([1 -2; -2 1]; label="a)")
diagonalize([1 -2; -2 3]; label="b)")
diagonalize([1.0 -2.0 0.0; -2.0 1.0 2.0; 0.0 2.0 0.0]; label="c)")

A_a = [1 -2; -2 1]
A_a^10

A_b = [5 -2; -2 5]
sqrt(A_b)
sqrt(A_b)^2

A_c = [1 -2; -2 1]
F = eigen(A_c)
cbrt_A = F.vectors * Diagonal(cbrt.(F.values)) * inv(F.vectors)
cbrt_A^3

A_d = [1 2; 2 3]
eigvals(A_d)
try
    display(sqrt(A_d))
catch e
    println("sqrt(A_d) □□ □□□□□□□□□□□: ", typeof(e))
end
```

```
F_d = eigen(A_d)
sqrt_A_d = F_d.vectors * Diagonal(sqrt.(Complex.(F_d.values))) * inv(
sqrt_A_d^2

A5 = [140  97  74 168 131;
       97 106  89 131  36;
       74  89 152 144  71;
      168 131 144  54 142;
      131  36  71 142  36]
eigvals(A5)
□5 = Diagonal(eigvals(A5))
L5 = LowerTriangular(A5)
@btime eigvals($A5);
@btime Diagonal(eigvals($A5));
@btime LowerTriangular($A5);

# --- 4.4.4. □□□□□□□□ □□□□□□ □□□□□□□□□ ---

function check_productive_def(A; label="")
    E = Matrix{Float64}(I, 2, 2)
    println(label, " A = ", A)
    for y in ([1.0, 0.0], [0.0, 1.0])
        x = (E - A) \ y
      println("  y = $y -> x = $x, □□□ x □ 0: ", all(x .>= 0))
    end
end

check_productive_def([1 2; 3 4]; label="a)")
```

```
check_productive_def([1 2; 3 4] / 2; label="b)")
check_productive_def([1 2; 3 4] / 10; label="c)")

function check_productive_inv(A; label="")
    E = Matrix{Float64}(I, size(A, 1), size(A, 1))
    M = inv(E - A)
    println(label, " (E-A)⁻¹ = ")
    display(M)
    println("   □□□ □□□□□□□□ □ 0: ", all(M .>= 0))
end

check_productive_inv([1 2; 3 1]; label="a)")
check_productive_inv([1 2; 3 1] / 2; label="b)")
check_productive_inv([1 2; 3 1] / 10; label="c)")

function check_productive_spec(A; label="")
    □ = eigvals(A)
    println(label, " □□□□□□□□□□□ □□□□□□□□: ", □)
    println("   □□□ |□| < 1: ", all(abs.(□) .< 1))
end

check_productive_spec([1 2; 3 1]; label="a)")
check_productive_spec([1 2; 3 1] / 2; label="b)")
check_productive_spec([1 2; 3 1] / 10; label="c)")
check_productive_spec([0.1 0.2 0.3; 0 0.1 0.2; 0 0.1 0.3]; label="d)")
```